

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра информационных технологий и автоматизированных систем

ОТЧЕТ
о лабораторной работе 3
Семестр: 4

На тему: «Реализация алгоритма Шимбелла»

Выполнил студент ИВТ-24-26:
Шишкин М.Г

(дата, подпись)

Проверил:

Рустамханова Г.И

(дата, подпись)

Пермь 2026

Содержание

1 Постановка задачи	3
2 Краткое описание решения	4
3 Код	5
4 Тестирование программы	10
5 Вывод	11
6 Github	12

1 Постановка задачи

Требуется разработать программу на языке C++, которая для ориентированного графа, заданного весовой матрицей размером 6×6 , находит веса всех путей заданной длины. На вход программа получает из файла матрицу весов (0 означает отсутствие ребра), затем запрашивает у пользователя направление поиска (минимум для кратчайших путей или максимум для длиннейших) и количество переходов K . На выходе программа выдаёт матрицу, где элемент $[i][j]$ содержит вес наилучшего пути из вершины i в вершину j за точно K переходов, либо INF или $-INF$, если такого пути не существует.

2 Краткое описание решения

В основе программы лежит алгоритм Шимбелла, который обобщает матричное умножение: обычные операции умножения и сложения заменяются на пару (сложение, взятие минимума) для задачи поиска кратчайших путей или (сложение, взятие максимума) для задачи поиска длиннейших путей. Таким образом, чтобы получить пути длины K , достаточно возвести исходную весовую матрицу в степень K в смысле min-plus или max-plus алгебры. Для повышения эффективности используется бинарное возведение в степень. При считывании матрицы из файла ноль интерпретируется как отсутствие ребра и заменяется на специальное значение INF (или NEG_INF для режима максимума), чтобы корректно обрабатывать недостижимые вершины.

3 Код

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <limits>
5
6  using namespace std;
7
8  const int N = 6;
9  const int INF = numeric_limits<int>::max() / 2;
10 const int NEG_INF = -numeric_limits<int>::max() / 2;
11
12 struct Matrix {
13     int data[N][N];
14
15     Matrix() {
16         for (int i = 0; i < N; ++i) {
17             for (int j = 0; j < N; ++j) {
18                 data[i][j] = INF;
19             }
20         }
21     }
22
23     void print(bool useMin = true) const {
24         for (int i = 0; i < N; ++i) {
25             for (int j = 0; j < N; ++j) {
26                 if (useMin && data[i][j] == INF) {
27                     cout << " INF\t";
28                 }
29                 else if (!useMin && data[i][j] == NEG_INF) {
30                     cout << "-INF\t";
31                 }
32                 else {
33                     cout << data[i][j] << "\t";
34                 }
35             }
36             cout << endl;
37         }
38     }
39 };
40
41 bool readMatrixFromFile(const string& filename, Matrix& mat) {
42     ifstream file(filename);
43     if (!file.is_open()) {
44         cerr << "Ошибка: не удалось открыть файл " << filename << endl;
45         return false;
46     }
47
48     for (int i = 0; i < N; ++i) {
49         for (int j = 0; j < N; ++j) {
```

```

49     for (int j = 0; j < N; ++j) {
50         int val;
51         if (!(file >> val)) {
52             cerr << "Ошибка: недостаточно данных в файле" << endl;
53             return false;
54         }
55         // 0 означает отсутствие ребра
56         if (val == 0) {
57             mat.data[i][j] = INF;
58         }
59         else {
60             mat.data[i][j] = val;
61         }
62     }
63 }
64
65 file.close();
66 return true;
67 }
68
69 // Преобразование матрицы для режима максимума (INF -> NEG_INF)
70 Matrix convertToMaxMode(const Matrix& mat) {
71     Matrix result;
72     for (int i = 0; i < N; ++i) {
73         for (int j = 0; j < N; ++j) {
74             if (mat.data[i][j] == INF) {
75                 result.data[i][j] = NEG_INF;
76             }
77             else {
78                 result.data[i][j] = mat.data[i][j];
79             }
80         }
81     }
82     return result;
83 }
84
85 Matrix shibellMin(const Matrix& A, const Matrix& B) {
86     Matrix C;
87     for (int i = 0; i < N; ++i) {
88         for (int j = 0; j < N; ++j) {
89             int best = INF;
90             for (int k = 0; k < N; ++k) {
91                 if (A.data[i][k] != INF && B.data[k][j] != INF) {
92                     int sum = A.data[i][k] + B.data[k][j];
93                     if (sum < best) {
94                         best = sum;
95                     }
96                 }
97             }

```

```

97         }
98         C.data[i][j] = best;
99     }
100 }
101 return C;
102 }
103
104 Matrix shimbellMax(const Matrix& A, const Matrix& B) {
105     Matrix C;
106     // Инициализируем все значения NEG_INF
107     for (int i = 0; i < N; ++i) {
108         for (int j = 0; j < N; ++j) {
109             C.data[i][j] = NEG_INF;
110         }
111     }
112
113     for (int i = 0; i < N; ++i) {
114         for (int j = 0; j < N; ++j) {
115             int best = NEG_INF;
116             for (int k = 0; k < N; ++k) {
117                 if (A.data[i][k] != NEG_INF && B.data[k][j] != NEG_INF) {
118                     int sum = A.data[i][k] + B.data[k][j];
119                     if (sum > best) {
120                         best = sum;
121                     }
122                 }
123             }
124             C.data[i][j] = best;
125         }
126     }
127     return C;
128 }
129
130 Matrix powerShimbell(const Matrix& base, int steps, bool useMin) {
131     if (steps == 0) {
132         Matrix result;
133         if (useMin) {
134             for (int i = 0; i < N; ++i) {
135                 for (int j = 0; j < N; ++j) {
136                     result.data[i][j] = (i == j) ? 0 : INF;
137                 }
138             }
139         }
140         else {
141             for (int i = 0; i < N; ++i) {
142                 for (int j = 0; j < N; ++j) {
143                     result.data[i][j] = (i == j) ? 0 : NEG_INF;
144                 }
145             }
146         }
147     }
148 }

```

```

145     }
146     return result;
147 }
148
149 Matrix result;
150 if (useMin) {
151     for (int i = 0; i < N; ++i) {
152         for (int j = 0; j < N; ++j) {
153             result.data[i][j] = (i == j) ? 0 : INF;
154         }
155     }
156 }
157 else {
158     for (int i = 0; i < N; ++i) {
159         for (int j = 0; j < N; ++j) {
160             result.data[i][j] = (i == j) ? 0 : NEG_INF;
161         }
162     }
163 }
164
165 Matrix power = base;
166 int exp = steps;
167
168 while (exp > 0) {
169     if (exp & 1) {
170         if (useMin) {
171             result = shinnellMin(result, power);
172         }
173         else {
174             result = shinnellMax(result, power);
175         }
176     }
177     if (useMin) {
178         power = shinnellMin(power, power);
179     }
180     else {
181         power = shinnellMax(power, power);
182     }
183     exp >>= 1;
184 }
185
186 return result;
187 }
188
189 int main() {
190     setlocale(LC_ALL, "Russian");
191     string filename;
192
193

```



```

193     string filename;
194     cout << "Введите имя файла с матрицей весов (6x6): ";
195     cin >> filename;
196
197     Matrix original;
198     if (!readMatrixFromFile(filename, original)) {
199         cerr << "Не удалось загрузить матрицу из файла. Программа завершена." << endl;
200         return 1;
201     }
202
203     cout << "\nИсходная матрица весов:" << endl;
204     original.print(true);
205
206     int choice;
207     cout << "\nВыберите направление поиска:" << endl;
208     cout << "1 - Кратчайшие пути (минимум)" << endl;
209     cout << "2 - Длиннейшие пути (максимум)" << endl;
210     cout << "Ваш выбор: ";
211     cin >> choice;
212
213     bool useMin;
214     Matrix workingMatrix;
215
216     if (choice == 1) {
217         useMin = true;
218         workingMatrix = original;
219         cout << "Выбраны кратчайшие пути (минимум)." << endl;
220     }
221     else if (choice == 2) {
222         useMin = false;
223         // Преобразуем матрицу для режима максимума
224         workingMatrix = convertToMaxMode(original);
225         cout << "Выбраны длиннейшие пути (максимум)." << endl;
226     }
227     else {
228         cerr << "Неверный выбор. Программа завершена." << endl;
229         return 1;
230     }
231
232     int steps;
233     cout << "Введите количество переходов (положительное целое число): ";
234     cin >> steps;
235
236     if (steps <= 0) {
237         cerr << "Количество переходов должно быть положительным. Программа завершена." << endl;
238         return 1;
239     }
240
241     // Вычисляем матрицу путей заданной длины
242     Matrix result = powerShinbell(workingMatrix, steps, useMin);
243
244     cout << "\nРезультат: матрица путей длиной " << steps << " переходов:" << endl;
245     if (useMin) {
246         cout << "(значение INF означает, что путь за " << steps << " переходов не существует)" << endl;
247         result.print(true);
248     }
249     else {
250         cout << "(значение -INF означает, что путь за " << steps << " переходов не существует)" << endl;
251         result.print(false);
252     }
253
254     return 0;
255 }

```

4 Тестирование программы

```
Введите имя файла с матрицей весов (6x6): g32.txt

Исходная матрица весов:
INF      INF      INF      3      INF      INF
1      INF      INF      INF      2      INF
INF      INF      INF      INF      INF      INF
6      INF      INF      INF      INF      INF
INF      2      INF      INF      INF      INF
INF      INF      INF      INF      4      INF

Выберите направление поиска:
1 - Кратчайшие пути (минимум)
2 - Длиннейшие пути (максимум)
Ваш выбор: 1
Выбраны кратчайшие пути (минимум).
Введите количество переходов (положительное целое число): 3

Результат: матрица путей длиной 3 переходов:
(значение INF означает, что путь за 3 переходов не существует)
INF      INF      INF      12      INF      INF
5      INF      INF      INF      6      INF
INF      INF      INF      INF      INF      INF
15      INF      INF      INF      INF      INF
INF      6      INF      6      INF      INF
7      INF      INF      INF      8      INF
```

5 Вывод

В ходе работы разработана программа, реализующая алгоритм Шимбелла для поиска кратчайших и длиннейших путей заданной длины в ориентированном графе. Программа корректно читает матрицу из файла, обрабатывает отсутствие ребер с помощью INF/-INF, поддерживает отрицательные веса.

6 Github

<https://github.com/MAKSPOWERO/math>